

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 8

Operating Systems

The Same Calls, Inside Your Own OS

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment and evidence 3

2.2 Part 1 - run inside xv6 3

2.3 Part 2 - pingpong 4

2.4 Optional - Prime sieve 5

3 Conclusions **6**

A Submitted source code **7**

A.1 user/run.c 7

A.2 user/runls.c 7

A.3 user/runmissing.c 8

A.4 user/pingpong.c 9

A.5 user/primes.c 10

1 Introduction

Repeat the Linux process lifecycle in xv6, implement a two-pipe byte exchange, and extend the pipe mechanism into a process-chain prime sieve.

This report follows the teacher's handout and includes every required Observe answer. Terminal-style listings replace new screenshots. Recorded output is identified with its execution environment; shortened excerpts are marked. Source files and build instructions are provided in the submission directory.

2 Tasks and results

2.1 Environment and evidence

Compiled and executed on 23 October 2026 with MIT xv6-riscv commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`, the RISC-V GNU compiler and QEMU. The recorded session used one emulated CPU and a combined image of the lab 04, 06 and 08 programs. Each submitted Makefile also built independently. The terminal blocks are recorded output, and `terminal-output.txt` contains the full session. PIDs and executable listings depend on the run.

2.2 Part 1 - run inside xv6

`user/run.c` preserves the supplied lab's program. The parent prints its PID and forks. The child prints its own PID, builds a NUL-terminated array of strings, and calls `exec("echo", args)`. The parent calls `wait(&status)` and reports the child's status. The Makefile includes `run` in `UPROGS`.

```
$ run
parent: my pid is 57
child: my pid is 58
hello from the child
parent: child 58 ended with status 0
```

2.2.1 The ls and missing-program variants

`user/runls.c` changes the child's array to `{ "ls", 0 }` and executes `ls`. `user/runmissing.c` changes the executable pathname to `nosuchprog`, which deliberately fails. Keeping separate variants allows the default `run` program to remain exactly the echo example and lets the assistant run all cases without editing a source file.

Recorded `runls` output, with its long directory listing shortened:

```
$ runls
parent: my pid is 59
child: my pid is 60
.              1 1 1024
..             1 1 1024
[remaining directory entries omitted]
parent: child 60 ended with status 0
```

```
$ runmissing
```

```
parent: my pid is 61
child: my pid is 62
exec failed
parent: child 62 ended with status 1
```

2.2.2 1. Compare the three calls with Linux

Operation	xv6 lab 08	Linux lab 07
Create the child	`fork()`	`fork()`
Replace the child image	`exec("echo", args)`	`execlp("ls", "ls", "-l", (char *)NULL)`
Reap the child	`wait(&status)`	`waitpid(pid, &status, 0)`

The lifecycle is the same: fork creates a separate process with inherited descriptors and memory contents; exec replaces the child image while retaining its PID; wait synchronizes with termination and reaps it. Linux `execlp` is a libc helper with PATH search and a variadic argument list. xv6's `exec` takes an explicit pathname and argument vector and does no PATH search. xv6 `wait` waits for a child and stores the exit value directly; Linux waitpid can select a child and returns an encoded status decoded with wait macros.

2.2.3 2. Who prints hello from the child

The `echo` program prints it. Before exec, `run` prints only the parent and child PID messages. Successful exec never returns to the old program; the new echo image uses the supplied argument strings to produce `hello from the child`.

2.2.4 3. Why the failure statuses differ

The xv6 branch after failed exec explicitly calls `exit(1)`, so its parent prints status 1. The Linux branch explicitly calls `exit(127)`, so its parent prints 127 after decoding it. The values were chosen by the programs; exec itself returns an error to the caller instead of automatically deciding this exit status. The xv6 parent still exits 0 after reporting its child, as in the supplied example.

2.3 Part 2 - pingpong

`user/pingpong.c` creates two pipes, `p2c` and `c2p`. The child closes `p2c[1]` and `c2p[0]`; the parent closes `p2c[0]` and `c2p[1]`. The parent initializes the byte to `p`, writes it to the first pipe, and waits in a read on the second pipe. The child reads the byte, prints its ping message, writes the byte back, closes its descriptors and exits. The parent receives the reply, prints pong, closes descriptors and reaps the child.

```
parent p2c[1] -- one byte --> p2c[0] child
parent c2p[0] <-- one byte -- c2p[1] child
```

```
$ pingpong
64: received ping
63: received pong
```

The actual child PID is 64 and the parent PID is 63. The order is guaranteed by the code: the child prints ping before writing the reply, and the parent prints pong only after receiving it. Scheduler timing may delay either process, but cannot reverse these messages in a successful exchange. Pipe, fork, read, write and wait errors are checked; a failed exchange exits nonzero.

2.3.1 1. Why two pipes

A pipe is a one-way byte stream. Using a separate pipe per direction gives each process an unambiguous input and output channel. If both processes read and write a single shared pipe, either can consume its own byte because a pipe does not address messages to particular readers. Two pipes implement the required reliable round trip without that race.

2.3.2 2. Which call waits for the reply

The parent's `read(c2p[0], &b, 1)` blocks while there are no bytes available and a writer remains open. The child writing its reply makes that read complete. The later `wait(&status)` reaps the child; it is not what makes the parent wait for the reply byte. Closing unused write ends also ensures a failed/exited peer can produce EOF rather than leaving a reader blocked indefinitely.

2.4 Optional - Prime sieve

`user/primes.c` generates integers 2 through 35 and builds a process chain. Each filter reads its first integer as a prime, prints it, and forwards only integers not divisible by that prime through a new pipe to its child. Reads and writes transfer one integer at a time. Every process closes inherited unused ends and its output end when input reaches EOF, then waits for its child. The empty final stage exits without printing.

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

The output matches all eleven primes in the inclusive interval 2 through 35. Closing the final writers propagates EOF through the chain so all filters can exit. Keeping only necessary descriptors also stays within xv6's per-process descriptor limit.

3 Conclusions

The xv6 process lifecycle follows the same fork-exec-wait pattern as Linux, with smaller interfaces and explicit executable paths. The failure status comes from the exit call chosen by the program. The two-pipe exchange guarantees ping before pong because the parent blocks for the reply. The optional sieve demonstrates how correct descriptor closure propagates EOF through a chain of processes.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 8: *The Same Calls, Inside Your Own OS*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/run.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.2 user/runls.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "ls", 0 };
    exec("ls", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.3 user/runmissing.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("nosuchprog", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

A.4 user/pingpong.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b = 'p';
  if(pipe(p2c) < 0){
    fprintf(2, "pingpong: pipe failed\n"); exit(1);
  }
  if(pipe(c2p) < 0){
    close(p2c[0]); close(p2c[1]);
    fprintf(2, "pingpong: pipe failed\n"); exit(1);
  }
  int pid = fork();
  if(pid < 0){
    close(p2c[0]); close(p2c[1]); close(c2p[0]); close(c2p[1]);
    fprintf(2, "pingpong: fork failed\n"); exit(1);
  }
  if(pid == 0){
    close(p2c[1]); close(c2p[0]);
    if(read(p2c[0], &b, 1) != 1){
      fprintf(2, "pingpong: read ping failed\n"); exit(1);
    }
    close(p2c[0]);
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1){
      fprintf(2, "pingpong: write pong failed\n"); exit(1);
    }
    close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);
  int failed = write(p2c[1], &b, 1) != 1;
  close(p2c[1]);
  if(!failed && read(c2p[0], &b, 1) == 1)
    printf("%d: received pong\n", getpid());
  else {
    fprintf(2, "pingpong: exchange failed\n"); failed = 1;
  }
  close(c2p[0]);
  int status;
  if(wait(&status) < 0 || status != 0)
    failed = 1;
  exit(failed);
}
```

A.5 user/primes.c

```c
#include "kernel/types.h"
#include "user/user.h"

/* One filter per prime. Mark noreturn so recursive process creation is clear. */
static void filter(int input) __attribute__((noreturn));

static void
filter(int input)
{
  int prime, value, p[2];
  int n = read(input, &prime, sizeof(prime));
  if(n == 0){ close(input); exit(0); }
  if(n != sizeof(prime)){ fprintf(2, "primes: read failed\n"); exit(1); }
  printf("prime %d\n", prime);
  if(pipe(p) < 0){ fprintf(2, "primes: pipe failed\n"); exit(1); }
  int pid = fork();
  if(pid < 0){ fprintf(2, "primes: fork failed\n"); exit(1); }
  if(pid == 0){ close(input); close(p[1]); filter(p[0]); }
  close(p[0]);
  while((n = read(input, &value, sizeof(value))) == sizeof(value)){
    if(value % prime && write(p[1], &value, sizeof(value)) != sizeof(value)){
      fprintf(2, "primes: write failed\n"); exit(1);
    }
  }
  close(input); close(p[1]);
  int status;
  if(wait(&status) < 0 || status != 0 || n != 0)
    exit(1);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int p[2];
  if(pipe(p) < 0){ fprintf(2, "primes: pipe failed\n"); exit(1); }
  int pid = fork();
  if(pid < 0){ fprintf(2, "primes: fork failed\n"); exit(1); }
  if(pid == 0){ close(p[1]); filter(p[0]); }
  close(p[0]);
  for(int value = 2; value <= 35; value++)
    if(write(p[1], &value, sizeof(value)) != sizeof(value)){
      fprintf(2, "primes: write failed\n"); exit(1);
    }
  close(p[1]);
  int status;
  if(wait(&status) < 0 || status != 0)
    exit(1);
  exit(0);
}
```